# Reporte de sprints y métricas ágiles

Programa de analítica predictiva: Sprints 1 y 2 con Scrum y XP

Cliente: FinBank S.A., Vicepresidencia de Riesgo y Analítica Código: GMAC-FB-2026-01

Daniel Felipe Martínez Hernández · Rafael Alejandro Baracaldo Doikov · Oscar Santiago Merino Suárez

## 1. Qué cubre este reporte

En el primer corte gestionamos los dos primeros sprints con PMBOK para el control y un tablero Kanban para la entrega, y los medimos con métricas de flujo. En este corte repetimos la misma simulación: el mismo backlog, las mismas 12 semanas, los mismos COP $180.000.000 del Charter y el mismo impedimento. Lo que cambia es la forma de trabajar. Usamos **Scrum** para la gobernanza (roles, eventos, compromiso y velocidad) y **Extreme Programming** para la parte técnica (programación en parejas, pruebas primero, integración continua y refactorización). Como vimos en clase, Scrum no dice cómo escribir ni probar el código, y esa es justamente la parte que cubre XP.

Roles: Rafael Baracaldo es el Product Owner, Oscar Merino es el Scrum Master y los tres somos Developers. Daniel Martínez sigue a cargo del reporte formal al patrocinador que exige el Charter.

## 2. Los tres pilares de Scrum en el proyecto

| Pilar | Qué significa | Dónde se vio en estos sprints |
|---|---|---|
| Transparencia | Lo importante del proceso está a la vista de quien decide. | Definition of Done escrita en el README; tasas de completitud por variable en el reporte de calidad; el bloqueo de T-05.5 reportado tal cual, sin ocultar los 5 puntos. |
| Inspección | Revisar los artefactos seguido para detectar desviaciones a tiempo. | Daily Scrum de 15 minutos; revisión de cada Pull Request por alguien que no escribió el código; el CI corre las pruebas en cada push. |
| Adaptación | Ajustar el plan apenas algo se sale de lo tolerable. | El día 7 del Sprint 2 el Product Owner reordenó el Sprint Backlog y se adelantó el target de mora; la Retrospectiva bajó el compromiso del Sprint 3 a 16 puntos. |

## 3. Cierre de los sprints

| Sprint | Fechas (2026) | Sprint Goal | Comprometido | Hecho | Meta |
|---|---|---|---|---|---|
| S1 | 10–21 ago | El proyecto queda gobernado y el equipo puede usar datos legalmente. | 16 | 16 | Cumplida |
| S2 | 24 ago–4 sep | Una base de datos validada para los modelos de mora y churn. | 20 | 15 | Parcial |

**Backlog del Sprint 1**

| ID | Ítem | Pts | Estado |
|---|---|---|---|
| US-01 | Charter aprobado por el patrocinador | 3 | Hecho |
| US-02 | Acuerdo de acceso a datos, Ley 1581 | 5 | Hecho |
| US-03 | Perfilamiento de clientes sobre la muestra anonimizada | 5 | Hecho |
| US-04 | Tablero, repositorio y Definition of Done | 3 | Hecho |

**Backlog del Sprint 2**

| ID | Ítem | Pts | Estado |
|---|---|---|---|
| T-05.0 | Script de extracción, datos de mora | 2 | Hecho |
| T-05.6 | Reglas de validación, con pruebas primero | 3 | Hecho |
| T-05.1 | Reporte de calidad de datos | 4 | Hecho |
| T-05.4 | Target de mora a 30 y 60 días | 3 | Hecho |
| T-05.2 | Diccionario de variables v1, validado por negocio | 3 | Hecho |
| T-05.5 | Extracto del historial de churn | 5 | Bloqueado |

El incremento al final del Sprint 2 es un pipeline que valida un extracto, genera el reporte de calidad, excluye 4 de las 22 variables candidatas por tener menos de 80 % de completitud, documenta las 18 restantes en el diccionario y construye los dos targets de mora. Corre con un solo comando desde el repositorio y lo cubren 16 pruebas automáticas. El hito M2 no se cierra, porque el diccionario todavía no tiene variables de churn.

## 4. Historias de usuario y criterios de aceptación

### 4.1. Revisión INVEST del backlog del Sprint 2

Antes del Planning revisamos cada ítem con los criterios INVEST adaptados a datos. Cinco de los seis pasaron. T-05.5 no cumplía la *I* (independiente): dependía de una credencial que maneja el área de IT y que no teníamos. En ese momento no teníamos una Definition of Ready que lo frenara, así que entró al sprint igual. Ese fue el origen del incumplimiento (sección 6).

| ID | Historia (formato Como / Quiero / Para) | I | N | V | E | S | T |
|---|---|---|---|---|---|---|---|
| T-05.6 | Como motor de scoring, quiero que el extracto se rechace si viola rangos, categorías o trae datos personales, para no entrenar con datos corruptos ni incumplir la Ley 1581. | Sí | Sí | Sí | Sí | Sí | Sí |
| T-05.1 | Como analista de riesgo, quiero saber la completitud de cada variable, para excluir con una razón escrita las que no sirven (riesgo R2). | Sí | Sí | Sí | Sí | Sí | Sí |
| T-05.4 | Como modelo de mora, quiero los targets a 30 y 60 días, para poder entrenar contra una variable objetivo formal. | Sí | Sí | Sí | Sí | Sí | Sí |
| T-05.5 | Como modelo de churn, quiero el historial de cancelaciones, para poder entrenar el modelo del objetivo O2. | No | Sí | Sí | Sí | Sí | Sí |

### 4.2. Criterios en formato Given / When / Then

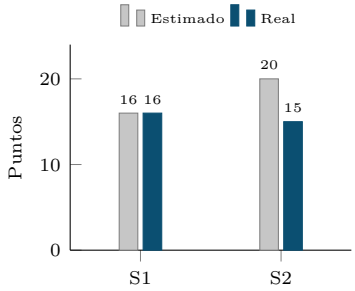
Los criterios de aceptación los acordamos con el Product Owner y los escribimos como escenarios. Cada escenario quedó automatizado como una prueba en `tests/`, así que pasa o falla sin discusión.

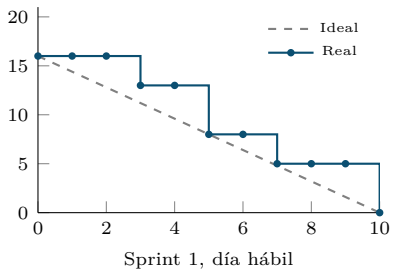
| Escenario | Prueba que lo verifica |
|---|---|
| **Dado** un extracto con un puntaje de buró de 1200, **cuando** se valida, **entonces** se reporta una violación *above_maximum* (el rango válido es 150–950). | `test_bureau_score_outside_range_is_reported` |
| **Dado** un extracto con valores faltantes en el puntaje, **cuando** se valida, **entonces** los faltantes no cuentan como fuera de rango. | `test_missing_values_are_not_range_violations` |
| **Dado** un extracto con una columna *cedula*, **cuando** se valida, **entonces** el proceso se detiene con error de dato personal. | `test_personal_data_column_is_rejected` |
| **Dada** una variable con exactamente 80 % de completitud, **cuando** se arma el reporte de calidad, **entonces** se conserva (el umbral es inclusivo). | `test_threshold_is_inclusive` |
| **Dada** una cuenta con 29 días de mora, **cuando** se calcula el target, **entonces** *default_30d* es 0; con 30 días es 1. | `test_targets_at_the_boundaries` |
| **Dada** una cuenta demasiado reciente para observarse, **cuando** se calcula el target, **entonces** queda como faltante y no como buen cliente. | `test_unobserved_accounts_stay_missing` |

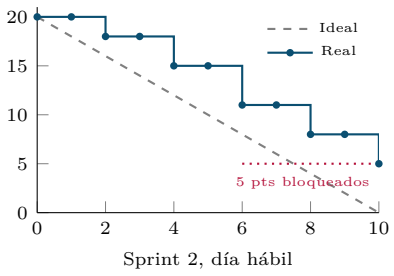
### 4.3. Definition of Ready y Definition of Done

| Definition of Ready (filtro de entrada, desde el Sprint 3) | Definition of Done (filtro de salida, vigente desde el Sprint 2) |
|---|---|
| Los datos de origen están accesibles con credenciales activas. | Reproducible desde el repositorio. |
| Criterios de aceptación escritos y aprobados por el Product Owner. | Pruebas en verde en el CI. |
| Estimación en puntos acordada por los Developers. | Pull Request aprobado por un compañero que no escribió el código. |
| Si hay mucha incertidumbre sobre los datos, se hizo un spike antes. | Validado por el dueño de negocio y mostrado en la Sprint Review. |

## 5. Velocidad y burndown



| Sprint | Estimado | Real | Real / estimado | Lectura |
|---|---|---|---|---|
| S1 | 16 | 16 | 100 % | Cuatro ítems cerrados. Los últimos 5 puntos se quemaron el día 10, cuando se firmó el acuerdo de acceso. |
| S2 | 20 | 15 | 75 % | Tomamos 4 puntos más de lo que acabábamos de demostrar que podíamos entregar, y 5 dependían de una credencial de IT. |
| Velocidad promedio | | 15,5 | | Capacidad para planear el Sprint 3. |

Las dos curvas muestran patrones que vimos en clase. En el Sprint 1 hay una meseta entre los días 7 y 9 y una caída al final: el acuerdo de acceso estaba esperando firma legal. En clase la caída final se asoció con revisiones de código acumuladas, pero aquí la causa fue externa al equipo. En el Sprint 2 la curva queda por encima de la ideal desde el día 2 y termina en una meseta de 5 puntos, que es justo el ítem bloqueado.

El faltante no fue por un equipo más lento. Los 15 puntos que dependían solo de nosotros se entregaron todos, al mismo ritmo del Sprint 1. La lección es de compromiso, no de esfuerzo.

## 6. Bitácora del Daily Scrum

Quince minutos a las 8:00, tres preguntas por persona. Solo se registran los días en que el plan cambió.

| Día | Hecho desde el último Daily | Plan del día | Impedimentos |
|---|---|---|---|
| S1 D1 | Sprint Planning: 4 ítems, 16 puntos. | Daniel redacta el Charter; Oscar arma el tablero; Rafael pide la muestra anonimizada. | Ninguno. |
| S1 D3 | Tablero y primera Definition of Done en uso (US-04). Muestra recibida. | Rafael perfila la muestra; Daniel envía el acuerdo al oficial de protección de datos. | El acceso a la bodega aún no está autorizado (R1); la muestra lo cubre. |
| S1 D5 | Perfilamiento de clientes cerrado (US-03). | Revisión del Charter con el patrocinador. | El script de perfilamiento solo existe en un portátil. |
| S1 D7 | Charter firmado (US-01). | Atender las observaciones del oficial de protección de datos. | Acuerdo esperando firma legal. |
| S1 D10 | Acuerdo firmado (US-02). Review y Retrospectiva. | Planning del Sprint 2. | Ninguno. |
| S2 D1 | Planning: 6 ítems, 20 puntos. Repositorio, CI y regla de Pull Request activos. | Rafael escribe el script de extracción; Oscar pide a IT la credencial de churn. | Ninguno. |
| S2 D3 | Script de extracción integrado (T-05.0). | Pairing 1, Rafael conduce y Daniel navega: reglas de validación escritas primero como pruebas que fallan. | Ninguno. |
| S2 D5 | Reglas de validación integradas, 7 pruebas en verde (T-05.6). | Reporte de calidad. Oscar empieza el refactor R-01. | Límites duplicados entre las reglas y el diccionario. |
| S2 D6 | Reporte de calidad en validación de negocio. R-01 integrado. | El extracto de churn no puede empezar. | IT no emite la credencial. T-05.5 bloqueado. |
| S2 D7 | El Scrum Master escala al patrocinador. El Product Owner reordena el Sprint Backlog. | Pairing 2, Oscar conduce y Rafael navega: se adelanta el target a 30 y 60 días. | Credencial pendiente, día 2. |
| S2 D8 | Reporte de calidad aceptado (T-05.1). Targets integrados (T-05.4). | Diccionario v1; sesión con cobranzas para validar variables. | Credencial pendiente, día 3. |
| S2 D10 | Diccionario v1 validado: 18 variables (T-05.2). | Sprint Review con el patrocinador y Retrospectiva. | Credencial pendiente, día 4. T-05.5 vuelve al Product Backlog. |

## 7. Retrospectivas

Las hicimos sin buscar culpables: el bloqueo de T-05.5 lo tratamos como una falla del proceso (no teníamos Definition of Ready), no como un error de una persona.

| | Empezar | Dejar de hacer | Continuar |
|---|---|---|---|
| S1 | Guardar todos los scripts en un repositorio compartido con CI. Escribir las reglas de datos como pruebas antes del análisis. Programar en pareja todo lo que codifique una regla de negocio. | Trabajar en portátiles personales. Aceptar un resultado porque su autor dice que corre. | Pedir una muestra anonimizada en paralelo a la solicitud de acceso: mantuvo el perfilamiento andando mientras R1 seguía abierto. |
| S2 | Comprometer como máximo la velocidad promedio. Aceptar un ítem solo si sus dependencias externas ya están resueltas (Definition of Ready). Reportar un bloqueo el mismo día en que aparece. | Comprometer puntos que dependen del calendario de otra área. Copiar los límites del diccionario dentro del código. | Pairing rotando el conductor. Pruebas antes que reglas. Reordenar el Sprint Backlog con el Product Owner en vez de esperar. |

| Origen | Acción de mejora | Responsable | Estado |
|---|---|---|---|
| S1 | Repositorio con CI; la Definition of Done exige pruebas en verde y Pull Request aprobado por un par. | Oscar | Hecho, S2 día 1 |
| S1 | Mínimo dos sesiones de pairing por sprint, registradas en la bitácora XP. | Rafael | Hecho en S2 |
| S2 | Compromiso del Sprint 3 limitado a 16 puntos. | Rafael | Aplicado en la sección 11 |
| S2 | Definition of Ready: ningún ítem con dependencia externa sin resolver entra a un sprint. | Oscar | Para el Planning del S3 |
| S2 | El Scrum Master escala los impedimentos en menos de 24 horas. | Oscar | Vigente |

## 8. Prácticas XP

| Práctica | Cómo la aplicamos | Evidencia en el repositorio |
|---|---|---|
| Programación en parejas | Dos sesiones en el Sprint 2. El conductor escribe y el navegante revisa supuestos y casos de borde; el conductor rotó entre sesiones. En la sesión 1 el navegante (Daniel) detectó que los faltantes no deben contar como fuera de rango; en la sesión 2 el navegante (Rafael) detectó que las cuentas no observadas deben quedar como faltantes. | Commits con `Co-authored-by`; `docs/XP_LOG.md`, práctica 01. |
| TDD (rojo, verde, refactor) | Primero se sube la prueba que falla (rojo), luego el código mínimo que la hace pasar (verde) y después se limpia sin cambiar el comportamiento (refactor). Así se escribieron las reglas de rangos, categorías, identificadores y datos personales. | 16 pruebas en `tests/`; resultado en el CI de cada Pull Request. |
| Integración continua y propiedad colectiva | Una rama por ítem, un Pull Request por rama, aprobado por alguien que no lo escribió; las pruebas corren en cada push. Cada miembro fue autor y revisor de al menos un PR. | Historial de Pull Requests en `main`; `.github/workflows/ci.yml`. |
| Diseño simple y refactorización | R-01 atacó un mal olor de *código duplicado*: cada variable tenía su propio bloque de validación y repetía un límite ya escrito en el esquema. Se extrajo una sola función que lee los límites del esquema. | `docs/refactoring/`; `docs/XP_LOG.md`, práctica 04. |

El refactor muestra cómo las prácticas se sostienen entre sí. Pudimos reestructurar el módulo con tranquilidad porque las 7 pruebas de la primera versión ya existían: pasaron sin editarse antes y después del cambio.

| R-01 | Antes | Después |
|---|---|---|
| Líneas del módulo | 99 | 84 |
| Variables validadas | 5 | 22 |
| Lugares donde se escribe un límite | 2 | 1 |
| Costo de agregar una variable | Un bloque de unas 10 líneas | Una línea en `schema.py` |

Repositorio: `https://github.com/OscarSantiagoMerino/DataForge-Consulting`

## 9. Diferencias con el primer corte

El Charter no cambió, ni el backlog, ni el impedimento. Cambió a qué se compromete el equipo, qué mide y qué acepta como terminado.

| | Primer corte: PMBOK y Kanban | Segundo corte: Scrum y XP |
|---|---|---|
| Unidad de trabajo | La tarjeta. Se jala trabajo cuando una columna tiene espacio bajo su límite WIP. El sprint es una compuerta de financiación. | El sprint. El Planning fija un Sprint Goal y los puntos a los que se compromete el equipo. |
| Roles | Líder de proyecto, analista de negocio y facilitador, con matriz RACI y un dueño por tarjeta. | Product Owner, Scrum Master y Developers. Cualquier miembro puede modificar cualquier archivo. |
| Métricas | Throughput, cycle time de 3,2 días, cumplimiento del WIP y variación de costo y cronograma. | Velocidad, estimado contra real y burndown. |
| Pregunta que responden | ¿Dónde está atascado el trabajo y seguimos dentro de la línea base? | ¿Cuánto podemos prometer en el siguiente sprint? |
| Impedimento | La tarjeta pasó a Bloqueado el día 6 y se escaló el día 7 por control de cambios, porque excedía la mitigación de R1. | Se reportó en el Daily del día 6; el Scrum Master escaló y el Product Owner reordenó el sprint a la mañana siguiente. |
| Terminado significa | Reproducible, documentado, validado por el dueño de negocio y mostrado en una revisión. | Lo mismo, más pruebas en verde en el CI y un Pull Request aprobado por un par. |
| Control de calidad | Revisión del dueño de negocio al final, en la columna de validación del cliente. | Incorporado antes de que el negocio lo vea: pruebas primero, pairing y revisión de pares. |
| Mejora | El patrocinador decide en la compuerta: continuar, redirigir o detener. | La compuerta sigue. El equipo suma su propia Retrospectiva con acciones con dueño. |

Notamos tres consecuencias. Primero, Kanban mostraba *que* una tarjeta estaba atascada; la velocidad mostró que el Sprint 2 estaba sobrecomprometido desde el primer día, algo que las métricas de flujo no decían. Segundo, la Definition of Done se volvió más exigente: los mismos 15 puntos ahora tienen pruebas y un segundo lector, y ninguna parte del pipeline depende de una sola persona. Tercero, Scrum nos dio menos libertad dentro del sprint. Con Kanban podíamos jalar la siguiente tarjeta en cualquier momento; aquí todo reordenamiento pasó por el Product Owner. La capa de PMBOK no desapareció: el Charter, la reserva y el control de cambios siguen enmarcando cada sprint.

## 10. Lo que haríamos distinto: el corte vertical

En clase se vio que dedicar sprints enteros a una sola capa (limpiar datos, luego variables, luego modelo) es un antipatrón, porque el usuario no recibe nada funcional durante semanas. Nuestro Sprint 2 se acerca a ese riesgo: el incremento es una base de datos validada, pero todavía no produce ninguna predicción. Lo justifica el Charter, que pone el diccionario como hito M2 antes del modelo, y el hecho de que el pipeline sí funciona de punta a punta desde el extracto hasta los targets. Aun así, para el Sprint 3 la meta es llegar pronto a un modelo base sencillo y complejizarlo después, en lugar de pulir cada capa por separado.

## 11. Pronóstico para el Sprint 3

Con una velocidad promedio de 15,5, el Sprint 3 se planea con 16 puntos como máximo. El Sprint Goal es el modelo de mora (US-06, 13 puntos), que protege el objetivo O1 en la semana 6. El extracto de churn (5 puntos) entra solo si la credencial existe en el Planning, como exige la nueva Definition of Ready. La lista semanal de alertas tempranas (US-07, 5 puntos) pasa al Sprint 4, que quedaría con 23 puntos frente a una capacidad de 16. Por eso el hito M4 de la semana 8 está en riesgo, y esa es la decisión que llevamos al patrocinador en la Sprint Review: liberar la credencial de IT esta semana o aceptar mover M4.